

FOUR RELATIONAL PROGRAMS^{*}

B. J. MacLennan
Computer Science Department
Naval Postgraduate School
Monterey, CA 93943

Abstract:

In this report we demonstrate the relational programming language RPL by using it to develop four programs. These programs are: (1) computing a table of word frequencies from a text; (2) a simple data processing example involving updating an employee file; (3) Gaussian elimination; and (4) minimizing deterministic finite state automata. We do not present the RPL language systematically; rather it is described incrementally through the examples.

1. Computing Word Frequencies

The first example, which is adapted from [MacLennan83], is to compute a frequency table F from a text (sequence of words) S . That is, given S such that $S \downarrow i$ is the i^{th} word, we compute F such that $F \downarrow w$ is the frequency (number of occurrences) of word w in S . For an example, we take $S = \langle \text{"to"}, \text{"be"}, \text{"or"}, \text{"not"}, \text{"to"}, \text{"be"} \rangle$, which is just an abbreviation for the relation:

$$S = \{1:\text{"to"}, 2:\text{"be"}, 3:\text{"or"}, 4:\text{"not"}, 5:\text{"to"}, 6:\text{"be"}\}$$

Pictorially,

1	"to"
2	"be"
3	"or"
4	"not"
5	"to"
6	"be"

In this case, the desired frequency table F is:

$$F = \{\text{"to"}:2, \text{"be"}:2, \text{"or"}:1, \text{"not"}:1\}$$

Pictorially,

"to"	2
"be"	2
"or"	1
"not"	1

In other words, "to" occurs twice, "be" occurs twice, "or" occurs once, and "not" occurs once. Of course, since F is a relation, the order in which the elements are listed is irrelevant.

To develop the general word-counting program, we work through this particular example. Since the frequency table is a function *from* the words *to* their frequencies, the first step is to reverse the columns of S :

$$S^{-1} = \{\text{"to"}:1, \text{"be"}:2, \text{"or"}:3, \text{"not"}:4, \text{"to"}:5, \text{"be"}:6\}$$

Pictorially,

^{*} The work reported herein was supported by Contract N00014-87-WR-24037 from the Office of Naval Research. Author's address after October 1987: Dept. Computer Science, Univ. of Tennessee, Knoxville, TN 37996-1301.

“to”	1
“be”	2
“or”	3
“not”	4
“to”	5
“be”	6

Notice that this relation is not a function (i.e., it is not single valued). We can make it a function by forming the “unit image” of the table:

$$\text{unimage } S^{-1} = \{ \text{“to”}:\{1, 5\}, \text{“be”}:\{2, 6\}, \text{“or”}:\{3\}, \text{“not”}:\{4\} \}$$

The **unimage** function is defined later. The relation $\text{unimage } S^{-1}$ can be pictured:

“to”	{1, 5}
“be”	{2, 6}
“or”	{3}
“not”	{4}

This tells us, for example, that the word “be” occurs in positions 2 and 6 in the text. We do not need to know the places where a given word occurs, but only the number of such places. Therefore, we compose the preceding table with the **size** (cardinality) function:

$$\text{unimage } S^{-1} \mid \text{size} = \{ \text{“to”}:2, \text{“be”}:2, \text{“or”}:1, \text{“not”}:1 \}$$

We have used the relative product operation to accomplish composition, $(f|g)x = g(fx)$. In effect this applies **size** to the right members of the preceding table, yielding:

“to”	2
“be”	2
“or”	1
“not”	1

This is the desired result. Notice that since **size** is defined for all sets, it is in effect an infinite relation; operations on both finite and infinite relations are permitted in RPL.

We have seen that we can compute the frequency table F for S by: $F \equiv \text{unimage } S^{-1} \mid \text{size}$. We can turn this into a function definition to compute the frequency table for any text S by:

$$\text{freq } S \equiv \text{unimage } S^{-1} \mid \text{size}$$

It remains to define **unimage**, which is not built into RPL. We use the builtin operator **unimg**, which is defined so that $T \text{ unimg } x$ is the set of all y such that the pair $x:y \in T$. This can be used to define **unimage**. To see this, note that the *left section* $[T \text{ unimg}]$ is the function that takes any x into its image under T . Although T is finite (and extensional), sections are always intensional, so it is necessary to convert $[T \text{ unimg}]$ to its extensional equivalent. This is accomplished with the RPL **restrict** operation, which converts an intensional relation to an extensional relation by restricting its domain to a finite set. Hence we define:

$$\text{unimage } T \equiv \text{dom } T \text{ restrict } [T \text{ unimg}]$$

The following is an example RPL session that defines the **freq** function and applies it to a particular text (‘?’ is the RPL prompt):

?> unimage $T \equiv \text{dom}T \text{ restrict } [T \text{ unimg}]$

?> freq $S \equiv \text{unimage } S^{-1} \mid \text{size}$

?> freq <“to”, “be”, “or”, “not”, “to”, “be”>

{“be”:2, “to”:2, “not”:1, “or”:1}

?> done

2. Employee File Update

Next we consider a simple data processing example adapted from [MacLennan83]. We are given an employee file F indexed by employee number. That is, $F \downarrow n$ is the record for employee number n . The employee records themselves are represented by functions from attribute names into attribute values. For example, if R is an employee record, then $R \downarrow \text{“N”}$ is the employee’s name, $R \downarrow \text{“R”}$ is his hourly rate, and $R \downarrow \text{“H”}$ is the hours worked this pay period. Here is an example employee file containing three records:

$$F \equiv \{124 : \{\text{“N”} : \text{“John”}, \text{“R”} : 10, \text{“H”} : 100\}, \\ 118 : \{\text{“N”} : \text{“Bill”}, \text{“R”} : 15, \text{“H”} : 120\}, \\ 207 : \{\text{“N”} : \text{“Sally”}, \text{“R”} : 14, \text{“H”} : 115\}\}$$

We are also given an update file U such that $U \downarrow n$ is the number of hours worked this week by employee number n . For example:

$$U \equiv \{118 : 6, 124 : 40, 207 : 40\}$$

Our task is to generate an updated employee file F' in which the hours worked (“H”) field has been updated for each employee.

First we define ‘sumhrs’ so that if R is an employee record and h is the hours worked this week, then $\text{sumhrs} \langle R, h \rangle$ is the new total hours. Clearly,

$$\text{sumhrs} \langle R, h \rangle \equiv (R \downarrow \text{“H”}) + h$$

Alternately, we can define this function in a variable-free style:

$$\text{sumhrs} \equiv [+]\circ([\downarrow \text{“H”}] \parallel \mathbf{I})$$

where the parallel construction operation ‘ $\parallel$ ’ is defined $(f \parallel g) \langle x, y \rangle = \langle fx, gy \rangle$. It is easy to see the two are equivalent:

$$\begin{aligned} \text{sumhrs} \langle R, h \rangle &= ([+]\circ([\downarrow \text{“H”}] \parallel \mathbf{I})) \langle R, h \rangle \\ &= [+] ([\downarrow \text{“H”}] \parallel \mathbf{I}) \langle R, h \rangle \\ &= [+] \langle [\downarrow \text{“H”}] R, \mathbf{I} h \rangle \\ &= (R \downarrow \text{“H”}) + h \end{aligned}$$

Our next task is to replace the old value of “H” field by h' . This can be accomplished by the *ordered union* operation ‘;’. For example, $\{\text{“H”} : h'\} ; R$ will return a record R' in which $R' \downarrow \text{“H”} = h'$ but all other fields of R' are the same as in R . How do we get the relation $\{\text{“H”} : h'\}$? Since this is just a (relational) sequence that’s equivalent to the array $\langle \text{“H”}, h' \rangle$, we can use the *as* operation to convert the array to a sequence. We solve for the function f that computes $\{\text{“H”} : h'\}$ from $\langle R, h \rangle$ as follows:

$$\begin{aligned}
f < R, h > &= \{ \text{"H"} : \text{sumhrs} < R, h > \} \\
&= \text{as} < \text{"H"}, \text{sumhrs} < R, h > > \\
&= \text{as} ([\text{"H"}], (\text{sumhrs} < R, h >)) \\
&= (\text{as} \circ [\text{"H"}], \circ \text{sumhrs}) < R, h >
\end{aligned}$$

Hence, $f \equiv \text{as} \circ [\text{"H"}], \circ \text{sumhrs}$.

It's necessary to get the corresponding records from the F and U files together so that they can be processed by f . This is accomplished by the extensional construction operation $\#$ defined so that $(F \# U) \downarrow n = \langle F \downarrow n, U \downarrow n \rangle$. With the given example files we have:

$$\begin{aligned}
F \# U &= \{ 124 : \langle \{ \text{"N"} : \text{"John"}, \text{"R"} : 10, \text{"H"} : 100 \}, 40 \rangle, \\
&\quad 118 : \langle \{ \text{"N"} : \text{"Bill"}, \text{"R"} : 15, \text{"H"} : 120 \}, 6 \rangle, \\
&\quad 207 : \langle \{ \text{"N"} : \text{"Sally"}, \text{"R"} : 14, \text{"H"} : 115 \}, 40 \rangle \}
\end{aligned}$$

Notice that the pairs $\langle F \downarrow n, U \downarrow n \rangle$ are just the inputs required for f . We combine the preceding results into a update file 'upd' defined so that $\text{upd} \downarrow n$ is $\{ \text{"H"} : h' \}$, representing the new hours worked field for employee number n . In this case,

$$\text{upd} = \{ 124 : \{ \text{"H"} : 140 \}, 118 : \{ \text{"H"} : 126 \}, 207 : \{ \text{"H"} : 155 \} \}$$

It's easy to solve for upd by using the relative product:

$$\text{upd} \downarrow n = f ((F \# U) \downarrow n) = ((F \# U) | f) \downarrow n$$

Hence, $\text{upd} = (F \# U) | f$. Substituting for f yields:

$$\text{upd} \equiv (F \# U) | (\text{as} \circ [\text{"H"}], \circ \text{sumhrs})$$

Now we're almost done. We want each record in F' to be the ordered union of the corresponding update record in upd and old record in F . Hence we solve:

$$\begin{aligned}
F' \downarrow n &= (\text{upd} \downarrow n) ; (F \downarrow n) \\
&= [;] < \text{upd} \downarrow n, F \downarrow n > \\
&= [;] ((\text{upd} \# F) \downarrow n) \\
&= ((\text{upd} \# F) | [;]) \downarrow n
\end{aligned}$$

Hence, $F' = (\text{upd} \# F) | [;]$. The complete session follows:

-- The Files

$$\begin{aligned}
F &\equiv \{ 124 : \{ \text{"N"} : \text{"John"}, \text{"R"} : 10, \text{"H"} : 100 \}, \\
&\quad 118 : \{ \text{"N"} : \text{"Bill"}, \text{"R"} : 15, \text{"H"} : 120 \}, \\
&\quad 207 : \{ \text{"N"} : \text{"Sally"}, \text{"R"} : 14, \text{"H"} : 115 \} \}
\end{aligned}$$

$$U \equiv \{ 118 : 6, 124 : 40, 207 : 40 \}$$

-- Computing the New File

$$\text{sumhrs} \equiv [+]\circ ([\downarrow \text{"H"}] \parallel \mathbf{I})$$

$$\text{upd} \equiv (F \# U) | (\text{as} \circ [\text{"H"}], \circ \text{sumhrs})$$

$$F' \equiv (\text{upd} \# F) | [;]$$

-- The New File

val F'

```
{124 : {“H” : 140, “N” : “John”, “R” : 10},
118 : {“H” : 126, “N” : “Bill”, “R” : 15},
207 : {“H” : 155, “N” : “Sally”, “R” : 14} }
```

(The val command prints the value of an identifier.) This result correctly reflects the fact that John (employee 124) has worked 140 hours, Bill (employee 118) has worked 126 hours, and Sally (employee 207) has worked 155 hours.

It is simple to modify the program so that it uses the input files OldMaster and Updates, and defines the output file NewMaster:

```
F ≡ file “OldMaster”
U ≡ file “Updates”
sumhrs ≡ [+] ∘ ([↓ “H”] ∥ I)
upd ≡ (F # U) | (as ∘ [“H” , ] ∘ sumhrs)
file “NewMaster” ≡ (upd # F) | [;]
```

3. Gaussian Elimination

The matrix is represented as a vector of vectors:

$$M = \begin{matrix} << & a_{11} & , \dots , & a_{1n} & , & b_1 & > , \\ & < & a_{21} & , \dots , & a_{2n} & , & b_2 & > , \\ & & \vdots & & \vdots & & \vdots & \\ & < & a_{n1} & , \dots , & a_{nn} & , & b_n & >> \end{matrix}$$

For the sake of simplicity we assume all the a_{ij} are nonzero. We use the operator ‘ $\downarrow$ ’ to select the an element of a vector. Thus ‘ $M \downarrow k$ ’ is the k th row of M and ‘ $M | [\downarrow k]$ ’ is the k th column of M .

The Gauss Elimination function will use n successive steps. Each of these steps will accomplish the transformation

$$< M, k > \mapsto < M', k + 1 >$$

where M' is obtained from M by performing the elimination process on the k th column:

$$M' = \text{elim} < M, k >$$

Thus the complete process is defined:

$$\text{Gauss } M = (\text{elim for } < 1, \dots, n >) M$$

Here we make use of the functional ‘ $(f \text{ for } S) x$ ’ which computes the sequence of values

$$\begin{aligned} y_1 &= f < x, S_1 > \\ y_2 &= f < y_1, S_2 > \\ &\vdots \\ y_n &= f < y_{n-1}, S_n > \end{aligned}$$

and returns y_n . Thef or functional is defined in terms of the builtin reduction operator (§) as follows:

$$f \text{ for } S \equiv [@ S] \circ [f \$]$$

Here @ is an explicit application operator, $f @ x = fx$. This definition off or can be understood by the expansion:

$$\begin{aligned}
(f \text{ for } S) x &= ([@ S] \circ [f \S]) x = [@ S] ([f \S] x) \\
&= [@ S] (f \S x) = (f \S x) @ S \\
&= (f \S x) S
\end{aligned}$$

The latter is the reduction from the left of array S by the function f with the starting value x :

$$(f \S x) S = f < f < \dots f < f < x, S \downarrow 1 >, S \downarrow 2 >, \dots, S \downarrow (n-1) >, S \downarrow n >$$

We turn now to the elimination process. We want $M' = \text{elim} < M, k >$, where M' results from M by zeroing all entries in column k of M , except the entry in row k , which is set to one. This can be accomplished by subtracting an appropriate matrix E from M :

$$M' = \text{matdif} < M, E >$$

Here matdif is a component-wise matrix difference function.

The matrix E is produced by multiplying the appropriate factors by the individual rows of M . For the first elimination step E is:

$$\begin{aligned}
E_1 &= \begin{aligned} &< (1 - 1/a_{11})M_1, \\ &(a_{21}/a_{11})M_1, \\ &\vdots \\ &(a_{n1}/a_{11})M_1 > \end{aligned}
\end{aligned}$$

The matrix resulting from subtracting E_1 from M is

$$\begin{aligned}
M_1 &= \begin{aligned} &<< 1, a_{12}', \dots a_{1n}', b_1' >, \\ &< 0, a_{22}', \dots a_{2n}', b_2' >, \\ &\vdots \\ &< 0, a_{n2}', \dots a_{nn}', b_n' >> \end{aligned}
\end{aligned}$$

At the next stage the elimination matrix is:

$$\begin{aligned}
E_2 &= \begin{aligned} &< (a_{12}'/a_{22}')M_2', \\ &(1 - 1/a_{22}')M_2', \\ &\vdots \\ &(a_{n2}'/a_{22}')M_2' > \end{aligned}
\end{aligned}$$

In general, if M'' is the matrix resulting from the $k-1$ st elimination step, then the elimination matrix for the k th step is

$$\begin{aligned}
E_k &= \begin{aligned} &< (a_{1k}''/a_{kk}'')M_k'', \\ &\vdots \\ &([a_{kk}'' - 1]/a_{kk}'')M_k'', \\ &\vdots \\ &(a_{nk}''/a_{kk}'')M_k'' > \end{aligned}
\end{aligned}$$

It is easy to see that E_k results from multiplying a vector V_k by the k th row of M'' . This is just the outer product of V_k and the k th row of M'' :

$$E_k = \text{outerprod} < V_k, M'' \downarrow k >$$

The vector V_k is

$$V_k = \begin{pmatrix} a_{1k}''/a_{kk}'' \\ \vdots \\ (a_{kk}'' - 1)/a_{kk}'' \\ \vdots \\ a_{nk}''/a_{kk}'' \end{pmatrix}$$

This is obtained by forming the scalar product of $1/a_{kk}''$ and the vector

$$U_k = \langle a_{1k}'', a_{2k}'', \dots, a_{kk}'' - 1, \dots, a_{nk}'' \rangle$$

This in turn is the result of subtracting from the k th column of M'' the *unit vector* $\text{unit} \langle M, k \rangle$, which has a 1 in the k th position, and a 0 in all others.

We now develop an explicit relational formula for V_k . For this purpose it will be convenient to treat it as a binary function $V_k = V \langle M, k \rangle$. This function is a scalar product,

$$\begin{aligned} V \langle M, k \rangle &= \text{scaprod} \langle 1 / \text{diag} \langle M, k \rangle, U_k \rangle \\ &= \text{scaprod} \langle 1 / \text{diag} \langle M, k \rangle, \text{vecdif} \langle \text{column} \langle M, k \rangle, \text{unit} \langle M, k \rangle \rangle \rangle \end{aligned}$$

The parameter $\langle M, k \rangle$ can be factored out by use of the intensional construction operation ‘ $\bar{}$ ’, which is defined so that $(f \bar{}, g \bar{})x = \langle f x, g x \rangle$. Factoring, we have:

$$\begin{aligned} V \langle M, k \rangle &= \text{scaprod} \langle 1 / \text{diag} \langle M, k \rangle, \text{vecdif} \langle \text{column} \langle M, k \rangle, \text{unit} \langle M, k \rangle \rangle \rangle \\ &= \text{scaprod} \langle ([1 /] \circ \text{diag}) \langle M, k \rangle, \text{vecdif} ((\text{column} \bar{} \text{unit}) \langle M, k \rangle) \rangle \\ &= \text{scaprod} \langle ([1 /] \circ \text{diag}) \langle M, k \rangle, (\text{vecdif} \circ (\text{column} \bar{} \text{unit})) \langle M, k \rangle \rangle \\ &= \text{scaprod} ((([1 /] \circ \text{diag}) \bar{} (\text{vecdif} \circ (\text{column} \bar{} \text{unit}))) \langle M, k \rangle) \\ &= (\text{scaprod} \circ (([1 /] \circ \text{diag}) \bar{} (\text{vecdif} \circ (\text{column} \bar{} \text{unit})))) \langle M, k \rangle \end{aligned}$$

Canceling $\langle M, k \rangle$ from both sides yields an explicit formula for V :

$$V \equiv \text{scaprod} \circ (([1 /] \circ \text{diag}) \bar{} (\text{vecdif} \circ (\text{column} \bar{} \text{unit})))$$

We proceed similarly to get a formula for elim :

$$\begin{aligned} \text{elim} \langle M, k \rangle &= \text{matdif} \langle M, E_k \rangle \\ &= \text{matdif} \langle M, \text{outerprod} \langle V_k, M \downarrow k \rangle \rangle \\ &= \text{matdif} \langle M, \text{outerprod} \langle V \langle M, k \rangle, M \downarrow k \rangle \rangle \end{aligned}$$

We perform some minor rearrangements so that $\langle M, k \rangle$ can be factored out of the right-hand side:

$$\begin{aligned} \text{elim} \langle M, k \rangle &= \text{matdif} \langle M, \text{outerprod} \langle V \langle M, k \rangle, [\downarrow] \langle M, k \rangle \rangle \rangle \\ &= \text{matdif} \langle M, (\text{outerprod} \circ (V \bar{} [\downarrow])) \langle M, k \rangle \rangle \\ &= \text{matdif} \langle [\downarrow 1] \langle M, k \rangle, (\text{outerprod} \circ (V \bar{} [\downarrow])) \langle M, k \rangle \rangle \\ &= (\text{matdif} \circ ([\downarrow 1] \bar{} (\text{outerprod} \circ (V \bar{} [\downarrow])))) \langle M, k \rangle \end{aligned}$$

Canceling $\langle M, k \rangle$ from both sides yields an explicit formula for elim :

$$\text{elim} \equiv \text{matdif} \circ ([\downarrow 1] \bar{} (\text{outerprod} \circ (V \bar{} [\downarrow])))$$

A complete RPL session demonstrating the Gaussian elimination function follows. Notice the simple definitions of the matrix and vector operations (`scaprod`, `matdif`, etc.).

-- Utility Functions

```

      con k ≡ λ x k
transmap f ≡ [ f ] ∘ [ # ]
  vecdif ≡ transmap [ - ]
scaprod < k, v > ≡ v | [ k × ]
outerprod < u, v > ≡ u | (scaprod ∘ [ , v ])
  matdif ≡ transmap vecdif
column < M, k > ≡ M | [ ↓ k ]
unit < M, k > ≡ < 1 , ... , size M > | [ [ = k ] → con 1; con 0 ]
diag < M, k > ≡ M ↓ k ↓ k
  f for S ≡ [ @ S ] ∘ [ f § ]

```

-- Gaussian Elimination

```

      V ≡ scaprod ∘ (([ 1.0 / ] ∘ diag) , (vecdif ∘ (column , unit)))
      elim ≡ matdif ∘ ([ ↓ 1 ] , (outerprod ∘ (V , [ ↓ ] )))
Gauss M ≡ (elim for < 1 , ... , size M >) M

```

-- Example Matrix

```

M ≡ << 3, 9, 33 > , < 2, -1, 1 >>

```

-- Execution

```

Gauss M

```

```

<< 1.0,  -2.38419E-7,  2.0 > ,
  < 0.0,      1.0,      3.0 >>

```

The matrix M represents the equations

$$\begin{aligned} 3x + 9y &= 33 \\ 2x - y &= 1 \end{aligned}$$

The result of **Gauss M** correctly reflects the solution $x = 2, y = 3$.

4. Minimization of Deterministic Finite Automata

The next example program is the equivalence and minimization of deterministic finite automata by an algorithm developed by Robert Floyd.¹ We assume that we have a finite alphabet Σ and a finite set Q of states. The set $F \subseteq Q$ represents the *final* (accepting) states. The finite relation T is such that for $a \in \Sigma$, $T \downarrow a$ is the transition relation for the symbol a . That is, $\langle q, q' \rangle \in T \downarrow a$ if and only if the symbol a takes state q into state q' .

Our goal is to define a relation R_∞ such that $\langle q, q' \rangle \in R_\infty$ if and only if q and q' are *not* equivalent states. This is done in a series of steps, starting from pairs of states that are known to be inequivalent, namely the final and nonfinal states ($\setminus$ is set difference):

$$R_0 \equiv F \times (Q \setminus F)$$

We now work backward: any states that under the same input lead to inequivalent states are themselves considered inequivalent. For example, R_0 relates inequivalent states; R_1 relates states related by R_0 together with those that under the same input character are taken into states related by R_0 ; R_2 relates states related by R_0 together with those that under the same one or two input characters lead to states related by R_0 ; and so on. Each step of this process is accomplished by a function ψ ; that is, we will define ψ so that $R_{i+1} = \psi R_i$. It will be easy to see that this process

1. Private communication, 1985.

converges in $n = (\text{size } Q)^2$ steps, so

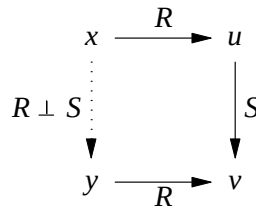
$$R_\infty \equiv \psi^n R_0$$

Next we consider ψ .

As a preliminary we define the *polymorphic image* of one relation under another relation. If R and S are two relations, then the polymorphic image under R of S , or more briefly the R -image of S , is defined

$$R \perp S \equiv R \mid S \mid R^{-1}$$

This has the following property: $R \perp S$ relates x to y if and only if there are u and v such that R relates x to u and y to v , and S relates u to v . That is, $\langle x, y \rangle \in R \perp S$ if and only if there are u and v such that $\langle x, u \rangle \in R$, $\langle y, v \rangle \in R$ and $\langle u, v \rangle \in S$. This can be visualized:



Now, if we have that R_i relates inequivalent states, then $(T \downarrow a) \perp R_i$ will relate those states that are carried by symbol a into states inequivalent by R_i . Thus, we define R_{i+1} so that it relates those states that are related by R_i together with those related by $(T \downarrow a) \perp R_i$, for any $a \in \Sigma$. Now, the right section $[\perp R_i]$ is the function that computes a polymorphic image of R_i , that is, $[\perp R_i] x = x \perp R_i$, so it is easy to see that

$$(T \mid [\perp R_i]) \downarrow a = [\perp R_i] (T \downarrow a) = (T \downarrow a) \perp R_i$$

Hence, the union of $(T \downarrow a) \perp R_i$, for all $a \in \Sigma$, is just the union of the range of the relation $T \mid [\perp R_i]$. This yields the definition of R_{i+1} and hence ψ :

$$R_{i+1} \equiv \psi R_i \equiv R_i \cup \bigcup (\text{rng } (T \mid [\perp R_i]))$$

This completes the definition of the inequivalence relation R_∞ . Two states are now equivalent if they are not inequivalent:

$$R_- \equiv Q^2 \setminus (R_\infty \cup R_\infty^{-1})$$

where Q^2 means $Q \times Q$.

The minimal machine is constructed on the basis of the equivalence classes of states under R_- . The equivalence class of a state q is just the unit image under R_- of q :

$$\text{eclass } q \equiv R_- \text{ unimg } q$$

The set of all such equivalence classes results from taking the image of Q under eclass :

$$Q_- \equiv \text{eclass img } Q$$

The img operation is defined so that $y \in f \text{ img } S$ if and only if $y = fx$ for some $x \in S$. In general, we define

$$\text{equiv} \equiv [\text{eclass img}],$$

so $Q_- \equiv \text{equiv } Q$. We take Q_- to be the states in the minimal machine.

It remains to construct the transition relation $T_{=}$ of the minimal machine. For all $\langle q, q' \rangle \in T \downarrow a$ we want

$$\langle \text{eclass } q, \text{eclass } q' \rangle \in T_{=} \downarrow a$$

Thus $T_{=} \downarrow a$ is the *homomorphic image* under eclass of $T \downarrow a$:

$$T_{=} \downarrow a = \text{eclass } (T \downarrow a)$$

In general, $\langle fx, fy \rangle \in f\R if and only if $\langle x, y \rangle \in R$. Thus $T_{=}$ is the (finite) composition of $[\text{eclass } \$]$ and T , which is the same as the (finite) relative product of T and $[\text{eclass } \$]$:

$$T_{=} \equiv T \mid [\text{eclass } \$]$$

The remainder of the minimal machine is easy to construct. For example, the final states of the new machine are just the equivalence classes of the original final states:

$$F_{=} \equiv \text{equiv } F$$

There follows the actual relational program to minimize a small automaton. It makes use of two auxiliary functions σ and ρ for defining the union of a set of sets:

-- DFA Minimization

-- Utility Functions

$$\begin{aligned} 1st &\equiv [\downarrow 1] \\ 2nd &\equiv [\downarrow 2] \\ r \perp s &\equiv r \mid s \mid r^{-1} \\ \sigma f &\equiv (f \circ (1st \mid (\varepsilon \circ 2nd))) \mid ([\downarrow] \circ (1 \mid (\text{un} \circ \varepsilon)) \circ 2nd)) \\ f \rho i &\equiv 1st \circ (\sigma f \text{ while } ([\neq \emptyset] \circ 2nd)) \circ [i,] \\ \cup &\equiv [\cup] \rho \emptyset \end{aligned}$$

-- Example 3 State DFA

$$\begin{aligned} \Sigma &\equiv \{1, 2\} \\ T &\equiv \{1 : \{10 : 10, 20 : 20\}, 2 : \{10 : 30, 20 : 30\}\} \\ Q &\equiv \{10, 20, 30\} \\ F &\equiv \{30\} \end{aligned}$$

-- Minimization

$$\begin{aligned} Q^2 &\equiv Q \times Q \\ n &\equiv \text{size } Q^2 \\ R_0 &\equiv F \times (Q \setminus F) \\ \psi R &\equiv R \cup \bigcup (\text{rng } (T \mid [\perp R])) \\ R_{\infty} &\equiv \psi^n R_0 \\ R_{=} &\equiv Q^2 \setminus (R_{\infty} \cup R_{\infty}^{-1}) \\ \text{eclass} &\equiv [R_{=} \text{ unimg}] \\ \text{equiv} &\equiv [\text{eclass img}] \\ Q_{=} &\equiv \text{equiv } Q \\ T_{=} &\equiv T \mid [\text{eclass } \$] \\ F_{=} &\equiv \text{equiv } F \end{aligned}$$

-- Minimized DFA

```
val Q=  
  { {10, 20}, {30} }  
val T=  
  { 1 : { {10, 20} : {10, 20} },  
    2 : { {10, 20} : {30} } }  
val F=  
  { {30} }
```

5. References

- [Brown&Mitton] Brown, J. R., and Mitton, S. J., *Relational Programming: Design and Implementation of a Prototype Interpreter*, MS thesis, Naval Postgraduate School, June 1985.
- [MacLennan83] MacLennan, B. J., "Relational Programming," Naval Postgraduate School Computer Science Department Technical Report NPS52-83-012, September 1983.